# SOFTWARE ENGINEERING
# 18CSC24

**T.Suvarna Kumari**
**Assistant Professor**
**Dept. of CSE**

# COURSE OBJECTIVES:

**The objectives of this course are**

- **To understand the Software Engineering Practice & Process Models.**

- **To understand Design Engineering, and Software Project Management.**

- **To gain knowledge of the overall project activities.**

# COURSE OUTCOMES:

**On Successful completion of this course, student will be able to**

- **Analyze** each module which gives the overall Software Engineering practice.
- **Enhance** the software project management skills.
- **Comprehend** the systematic methodologies involved in Software Engineering.
- **Design** and develop a software product in accordance with Software Engineering principles.
- **Formulate** the skills necessary for independently developing a complete software project.

# INTRODUCTION

- **The world can't operate without software.**

- **Industries are controlled by software systems, as the financial systems, scientific labs, infrastructures and utilities, games, film, television, and the list goes on.**

***"So, what's a software anyway?" …***

# SOFTWARE:

- **A software is a computer programs along with the associated documents and the configuration data that make these programs operate correctly.**

- ***A program is a set of instructions (written in form of human-readable code) that performs a specific task.***

# TYPES OF SOFTWARE SYSTEMS:

- There are many different types of software systems from simple to complex systems.

- These systems may be developed for a particular customer, like systems to support a particular business process, or developed for a general purpose, like any software for our computers such as word processors.

- Many systems are now being built with a generic product as base, which is then adapted to suit the requirements of a customer such as SAP system.

# SUCCESSFUL SOFTWARE SYSTEM:

- A good software should deliver the main required functionality.

- Other set of attributes — called quality or non-functional — should be also delivered.

- Examples of these attributes are, the software is written in a way that can be adapted to changes, response time, performance (less use of resources such as memory and processor time), usable; acceptable for the type of the user it's built for, reliable, secure, safe, …etc.

# SOFTWARE ENGINEERING:

- Software engineering is an engineering discipline that's applied to the development of software in a *systematic* approach (called a software process).

- It's the application of theories, methods, and tools to design, build a software that meets the specifications efficiently, cost-effectively, and ensuring quality.

- It's not only concerned with the technical process of building a software, it also includes activities to manage the project, develop tools, methods and theories that support the software production.

# *Contd..*

- ***Not applying software engineering methods results in more expensive, less reliable software, and it can be vital on the long term, as the changes come in, the costs will dramatically increase.***

- **Different methods and techniques of software engineering are appropriate for different types of systems.**

- **For example, games should be developed using series of prototypes, while critical control systems require a complete analyzable specification to be developed.**

## computer science vs software engineering

- **Computer science focuses on the theory and fundamentals, like algorithms, programming languages, theories of computing, artificial intelligence, and hardware design.**

- **Software engineering is concerned with the activities of developing and managing a software.**

# SOFTWARE ENGINEERS:

•The job of a software engineer is difficult.

• They has to balance between different people involved, such as:

**Dealing with users:** User don't know what to expect exactly from the software. The concern is always about the ease of use and response time.

**Dealing with technical people:** Developers are more technically inclined people so they think more of database terms, functionality, etc.

**Dealing with management:** They are concerned with return on their investment(ROI), and meeting the schedule.

- **For success in large software developments, it is important to follow an engineering approach, consisting of a well defined process.**

- **That's what we're going to explore next in the "Software Processes".**

# Chapter 1

- **Software & Software Engineering**

# What is Software?

***Software is:***

*(1) instructions (computer programs) that* ***when executed provide desired features****,* ***function****, and* ***performance****;*

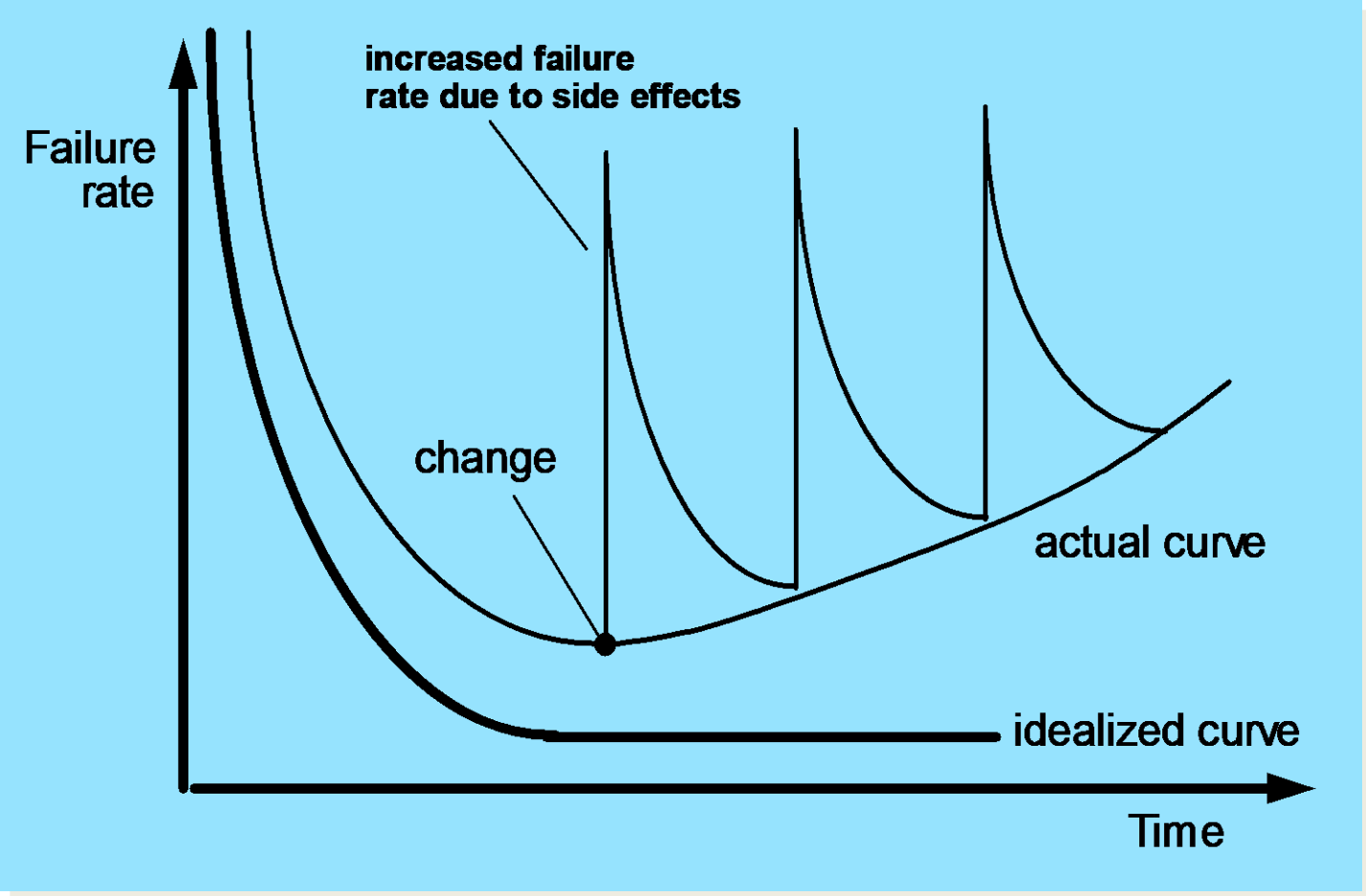
*(2) data structures that* ***enable*** *the programs to* ***adequately manipulate information*** *and*

*(3) documentation that* ***describes*** *the* ***operation*** *and* ***use of the programs.***

# What is Software?

- ***Software is developed or engineered, it is not manufactured in the classical sense.***
- ***Software doesn't "wear out."***
- ***Although the industry is moving toward component-based construction, most software continues to be custom-built.***

# Wear vs. Deterioration



# Software Applications

- system software
- application software
- engineering/scientific software
- embedded software
- product-line software
- WebApps (Web applications)
- AI software

# Software—New Categories

- Open world computing—pervasive, distributed computing
- Ubiquitous computing—wireless networks
- Netsourcing—the Web as a computing engine
- Open source—"free" source code open to the computing community (a blessing, but also a potential curse!)
- Also …
  - Data mining
  - Grid computing
  - Cognitive machines
  - Software for nanotechnologies

# Legacy Software

## *Why must it change?*

- software must be adapted to meet the needs of new computing environments or technology.
- software must be enhanced to implement new business requirements.
- software must be extended to make it interoperable with other more modern systems or databases.
- software must be re-architected to make it viable within a network environment.

# Characteristics of WebApps - I

- **Network intensiveness. A WebApp resides on a network and must serve the needs of a diverse community of clients.**
- **Concurrency. A large number of users may access the WebApp at one time.**
- **Unpredictable load. The number of users of the WebApp may vary by orders of magnitude from day to day.**
- **Performance. If a WebApp user must wait too long (for access, for server-side processing, for client-side formatting and display), he or she may decide to go elsewhere.**
- **Availability. Although expectation of 100 percent availability is unreasonable, users of popular WebApps often demand access on a "24/7/365" basis.**

# Characteristics of WebApps - II

- **Data driven. The primary function of many WebApps is to use hypermedia to present text, graphics, audio, and video content to the end-user.**
- **Content sensitive. The quality and aesthetic nature of content remains an important determinant of the quality of a WebApp.**
- **Continuous evolution. Unlike conventional application software that evolves over a series of planned, chronologically-spaced releases, Web applications evolve continuously.**
- **Immediacy. Although *immediacy*—the compelling need to get software to market quickly—is a characteristic of many application domains, WebApps often exhibit a time to market that can be a matter of a few days or weeks.**
- **Security. Because WebApps are available via network access, it is difficult, if not impossible, to limit the population of end-users who may access the application.**
- **Aesthetics. An undeniable part of the appeal of a WebApp is its look and feel.**

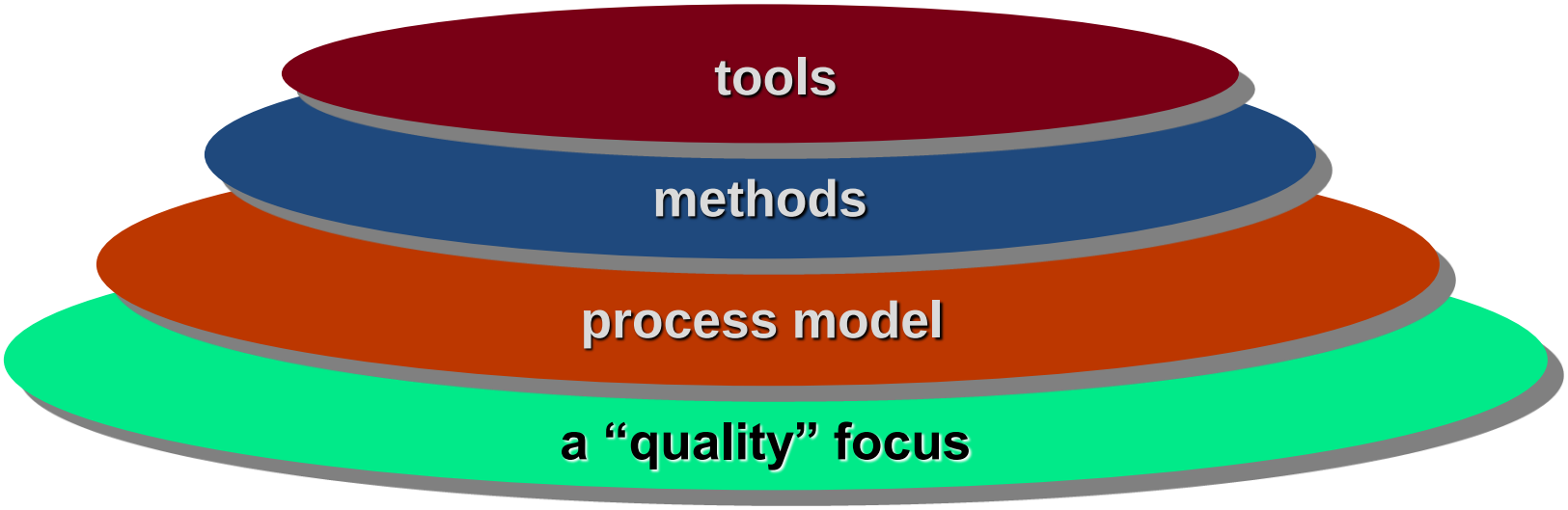
# Software Engineering

- **Some realities:**
  - *a concerted effort should be made to understand the problem before a software solution is developed.*
  - *design becomes a pivotal activity.*
  - *software should exhibit high quality.*
  - *software should be maintainable.*
- **The seminal definition:**
  - *[Software engineering is] the establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines.*

# Software Engineering

- **The IEEE definition:**
  - ***Software Engineering:***
  - ***(1) The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.***
  - ***(2) The study of approaches as in (1).***

# A Layered Technology




*Software Engineering*

# A Process Framework

**Process framework**

**Framework activities**

- work tasks
- work products
- milestones & deliverables
- QA checkpoints

**Umbrella Activities**

# Framework Activities

- Communication
- Planning
- Modeling
  - Analysis of requirements
  - Design
- Construction
  - Code generation
  - Testing
- Deployment

# Umbrella Activities

- Software project management
- Formal technical reviews
- Software quality assurance
- Software configuration management
- Work product preparation and production
- Reusability management
- Measurement
- Risk management

# Adapting a Process Model

- the overall flow of activities, actions, and tasks and the interdependencies among them
- the degree to which actions and tasks are defined within each framework activity
- the degree to which work products are identified and required
- the manner which quality assurance activities are applied
- the manner in which project tracking and control activities are applied
- the overall degree of detail and rigor with which the process is described
- the degree to which the customer and other stakeholders are involved with the project
- the level of autonomy given to the software team
- the degree to which team organization and roles are prescribed

# The Essence of Practice

- **Polya suggests:**
  1. ***Understand the problem*** **(communication and analysis).**
  2. ***Plan a solution*** **(modeling and software design).**
  3. ***Carry out the plan*** **(code generation).**
  4. ***Examine the result for accuracy*** **(testing and quality assurance).**

# 1.Understand the Problem

- ***Who has a stake in the solution to the problem?*** **That is, who are the stakeholders?**

- ***What are the unknowns?*** **What data, functions, and features are required to properly solve the problem?**

- ***Can the problem be compartmentalized?*** **Is it possible to represent smaller problems that may be easier to understand?**

- ***Can the problem be represented graphically?*** **Can an analysis model be created?**

# 2.Plan the Solution

- ***Have you seen similar problems before?*** **Are there patterns that are recognizable in a potential solution? Is there existing software that implements the data, functions, and features that are required?**

- ***Has a similar problem been solved?*** **If so, are elements of the solution reusable?**

- ***Can subproblems be defined?*** **If so, are solutions readily apparent for the subproblems?**

- ***Can you represent a solution in a manner that leads to effective implementation?*** **Can a design model be created?**

# 3.Carry Out the Plan

- ***Does the solution conform to the plan?*** **Is source code traceable to the design model?**

- ***Is each component part of the solution provably correct?*** **Has the design and code been reviewed, or better, have correctness proofs been applied to algorithm?**

# 4.Examine the Result

- ***Is it possible to test each component part of the solution?*** **Has a reasonable testing strategy been implemented?**

- ***Does the solution produce results that conform to the data, functions, and features that are required?*** **Has the software been validated against all stakeholder requirements?**

# Hooker's General Principles

- **1:** ***The Reason It All Exists***
- **2:** ***KISS (Keep It Simple, Stupid!)***
- **3:** ***Maintain the Vision***
- **4:** ***What You Produce, Others Will Consume***
- **5:** ***Be Open to the Future***
- **6:** ***Plan Ahead for Reuse***
- **7***: Think!*

# Software Myths

- Affect managers, customers (and other non-technical stakeholders) and practitioners.
- Are believable because they often have elements of truth,

*but …*

- Invariably lead to bad decisions,

*therefore …*

- Insist on reality as you navigate your way through software engineering.

# How It all Starts

- ***SafeHome:***
  - **Every software project is precipitated by some business need—**
    - **the need to correct a defect in an existing application;**
    - **the need to the need to adapt a 'legacy system' to a changing business environment;**
    - **the need to extend the functions and features of an existing application, or**
    - **the need to create a new product, service, or system.**

# SOFTWARE MYTHS-3

1.Management myths(4)

2.Customer myths(2)

3.Practitioner's myths(4)

# Management myths(4)

**Myth 1:** We already have a book that's full of standards and procedures for building software, won't that provide my people with everything they need to know?

**Reality:**

- The book of standards may very well exist, but is it used?
- Are software practitioners aware of its existence?
- Does it reflect modern software engineering practice?
- Is it complete?
- Is it streamlined to improve time to delivery while still maintaining a focus on quality?
- In many cases, the answer to all of these questions is "no."

**Myth 2:** My people have state-of-the-art software development tools, after all, we buy them the newest computers.

**Reality:**

- It takes much more than the latest model mainframe, workstation, or PC to do high-quality software development.

- Computer-aided software engineering (CASE) tools are more important than hardware for achieving good quality and productivity, yet the majority of software developers still do not use them effectively.

**Myth 3:** If we get behind schedule, we can add more programmers and catch up.

**Reality:**

- Software development is not a mechanistic process like manufacturing.
- In the words of Brooks: "adding people to a late software project makes it later.“
- As new people are added, people who were working must spend time educating the newcomers, thereby reducing the amount of time spent on productive development effort.
- People can be added but only in a planned and well-coordinated manner.

**Myth 4:** If I decide to outsource the software project to a third party, I can just relax and let that firm build it.

**Reality:**

- If an organization does not understand how to manage and control software projects internally, it will invariably struggle when it outsources software projects.

**Customer myths(2)-** a person at the next desk, a technical group down the hall, the marketing/sales department, or an outside company that has requested software under contract.

**Myth 1:** A general statement of objectives is sufficient to begin writing programs— we can fill in the details later.

**Reality:**

- A poor up-front definition is the major cause of failed software efforts.
- A formal and detailed description of the information domain, function, behavior, performance, interfaces, design constraints, and validation criteria is essential.
- These characteristics can be determined only after thorough communication between customer and developer.

**Myth 2:** Project requirements continually change, but change can be easily accommodated because software is flexible.

**Reality:**

- It is true that software requirements change, but the impact of change varies with the time at which it is introduced.
- Early requests for change can be accommodated easily.
- The customer can review requirements and recommend modifications with relatively little impact on cost.
- When changes are requested during software design, the cost impact grows rapidly.

- Resources have been committed and a design framework has been established.
- Change can cause upheaval that requires additional resources and major design modification, that is, additional cost.
- Changes in function, performance, interface, or other characteristics during implementation (code and test) have a severe impact on cost.
- Change, when requested after software is in production, can be over an order of magnitude more expensive than the same change requested earlier.

## Practitioner's myths(4)

**Myth 1:** Once we write the program and get it to work, our job is done.

**Reality:**

- Someone once said that "the sooner you begin 'writing code', the longer it'll take you to get done."

- Industry data indicate that between 60 and 80 percent of all effort expended on software will be expended after it is delivered to the customer for the first time.

**Myth 2:** Until I get the program "running" I have no way of assessing its quality.

**Reality:**

- One of the most effective software quality assurance mechanisms can be applied from the inception of a project—the formal technical review.

- Software reviews are a "quality filter" that have been found to be more effective than testing for finding certain classes of software defects.

**Myth 3:** The only deliverable work product for a successful project is the working program.

**Reality:**

- A working program is only one part of a software configuration that includes many elements.

- Documentation provides a foundation for successful engineering and, more important, guidance for software support.

**Myth 4:** Software engineering will make us create voluminous and unnecessary documentation and will invariably slow us down.

**Reality:**

- Software engineering is not about creating documents. It is about creating quality.

- Better quality leads to reduced rework. And reduced rework results in faster delivery times.

# Process Models:

**Process framework has activities such as:**
**communication,**
**planning,**
**modeling,**
**construction,**
**deployment.**

# Perspective/conventional process models:

- We call them perspective because they prescribe a set of process elements-

framework activities,
SE actions,
tasks,
work products,
quality assurance and
change control mechanisms.

# Classical Waterfall Model

- Classical waterfall model is the basic software development life cycle model.
- It is very simple but idealistic.
- Earlier this model was very popular but nowadays it is not used.
- But it is very important because all the other software development life cycle models are based on the classical waterfall model.

•Classical waterfall model divides the life cycle into a set of phases.

• This model considers that one phase can be started after completion of the previous phase.

•That is the output of one phase will be the input to the next phase.

•Thus the development process can be considered as a sequential flow in the waterfall.

•Here the phases do not overlap with each other.

## Advantages of Classical Waterfall Model

- idealistic model for software development.
- very simple, so it can be considered as the basis for other software development life cycle models.

Below are some of the major advantages of this SDLC model:

- This model is very simple and is easy to understand.
- Phases in this model are processed one at a time.
- Each stage in the model is clearly defined.
- This model has very clear and well understood milestones.

- Process, actions and results are very well documented.
- Reinforces good habits: define-before- design, design-before-code.
- works well for smaller projects and projects where requirements are well understood.

# Drawbacks of Classical Waterfall Model

- suffers from various shortcomings, i.e we can't use it in real projects, but we use other software development lifecycle models which are based on the classical waterfall model.

Below are some major drawbacks of this model:

**No feedback path**

**Difficult to accommodate change requests**

**No overlapping of phases**

# Evolutionary Models

- **Evolutionary Models are iterative.**

**Types:**

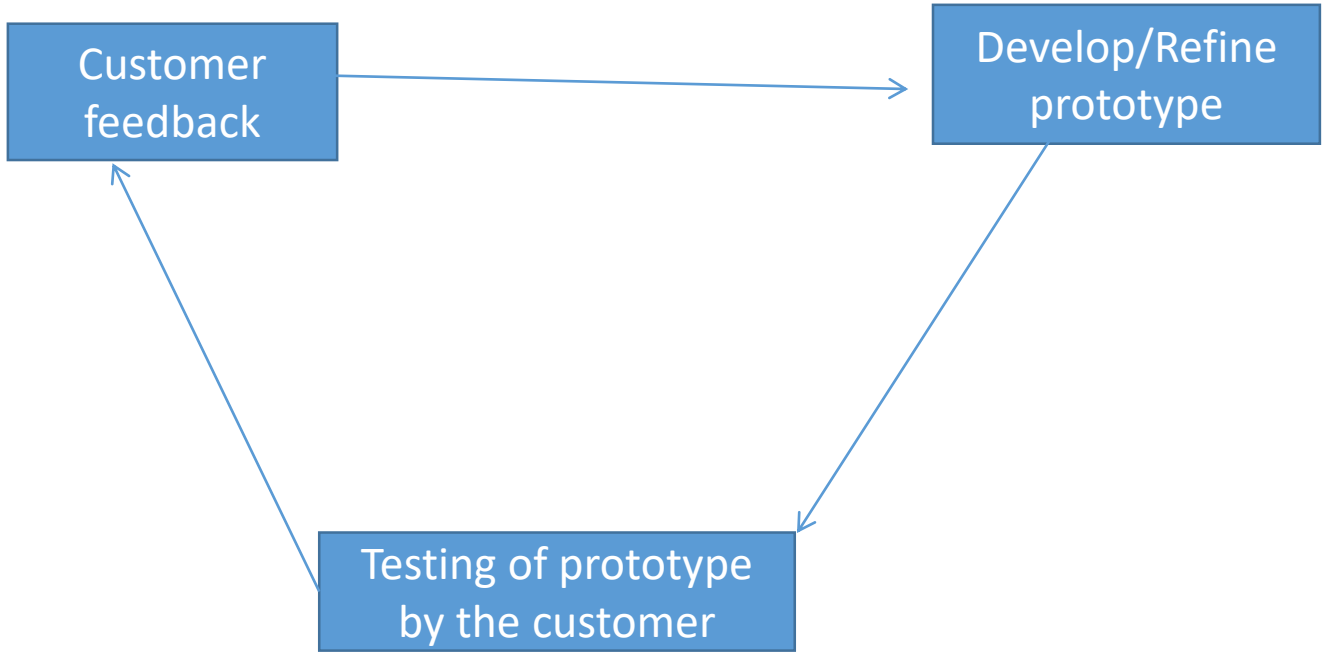
**1.Prototyping.**

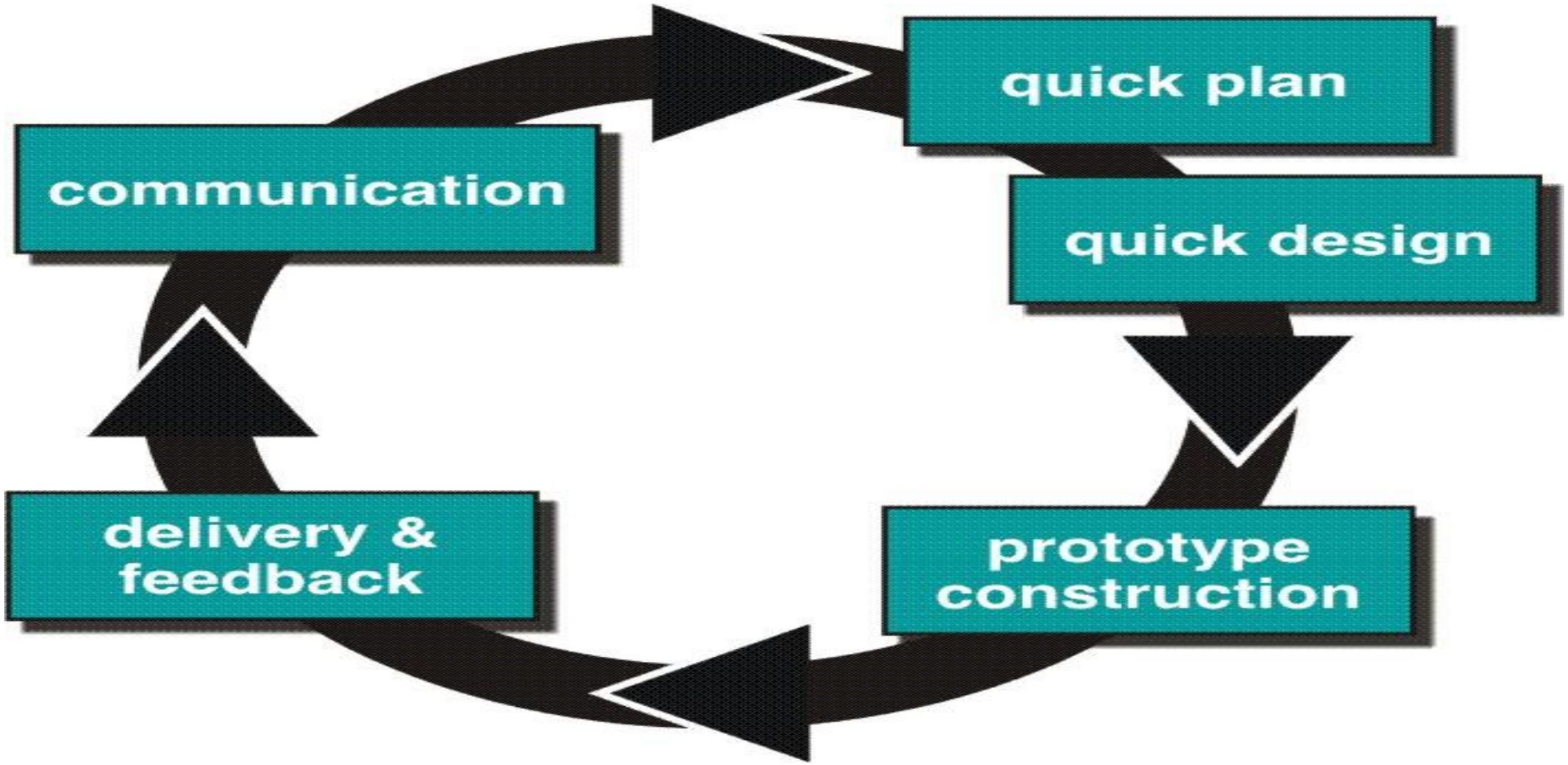
**2.The spiral model.**

**3.The concurrent development model.**

# 1.Prototyping:

- Process of developing a working replication of a product or system.
- Customer f/b is used to develop/refine prototype which is used for testing and it further results in customer feedback.




communication
quick plan
quick design
prototype construction
delivery & feedback

- One of the most popularly used Software Development Life Cycle Models (SDLC models).

- Used when the customers do not know the exact project requirements beforehand.

- A prototype of the end product is first developed, tested and refined as per customer feedback repeatedly till a final acceptable prototype is achieved which forms the basis for developing the final product.

# Advantages :

•Customers get to see the partial product early in the life cycle. This ensures a greater level of customer satisfaction and comfort.

• New requirements can be easily accommodated as there is scope for refinement.

• Missing functionalities can be easily figured out.

• Errors can be detected much earlier *thereby saving a lot of effort and cost, besides enhancing the quality of the software.*

• The developed prototype can be reused by the developer for more complicated projects in the future.

• Flexibility in design.

## Disadvantages :

- *Costly w.r.t time as well as money.*

- Too much variation in requirements each time the prototype is evaluated by the customer.

- Poor Documentation due to continuously changing customer requirements.

- very difficult for the developers to accommodate all the changes demanded by the customer.

- uncertainty in determining the number of iterations that would be required before the prototype is finally accepted by the customer.

- After seeing an early prototype, the customers sometimes demand the actual product to be delivered soon.

- Developers in a hurry to build prototypes may end up with sub-optimal solutions.

- The customer might lose interest in the product if he/she is not satisfied with the initial prototype.

# Use :

- The Prototyping Model should be used when the requirements of the product are not clearly understood or are unstable.

- Also be used if requirements are changing quickly.

# 2.The spiral model:

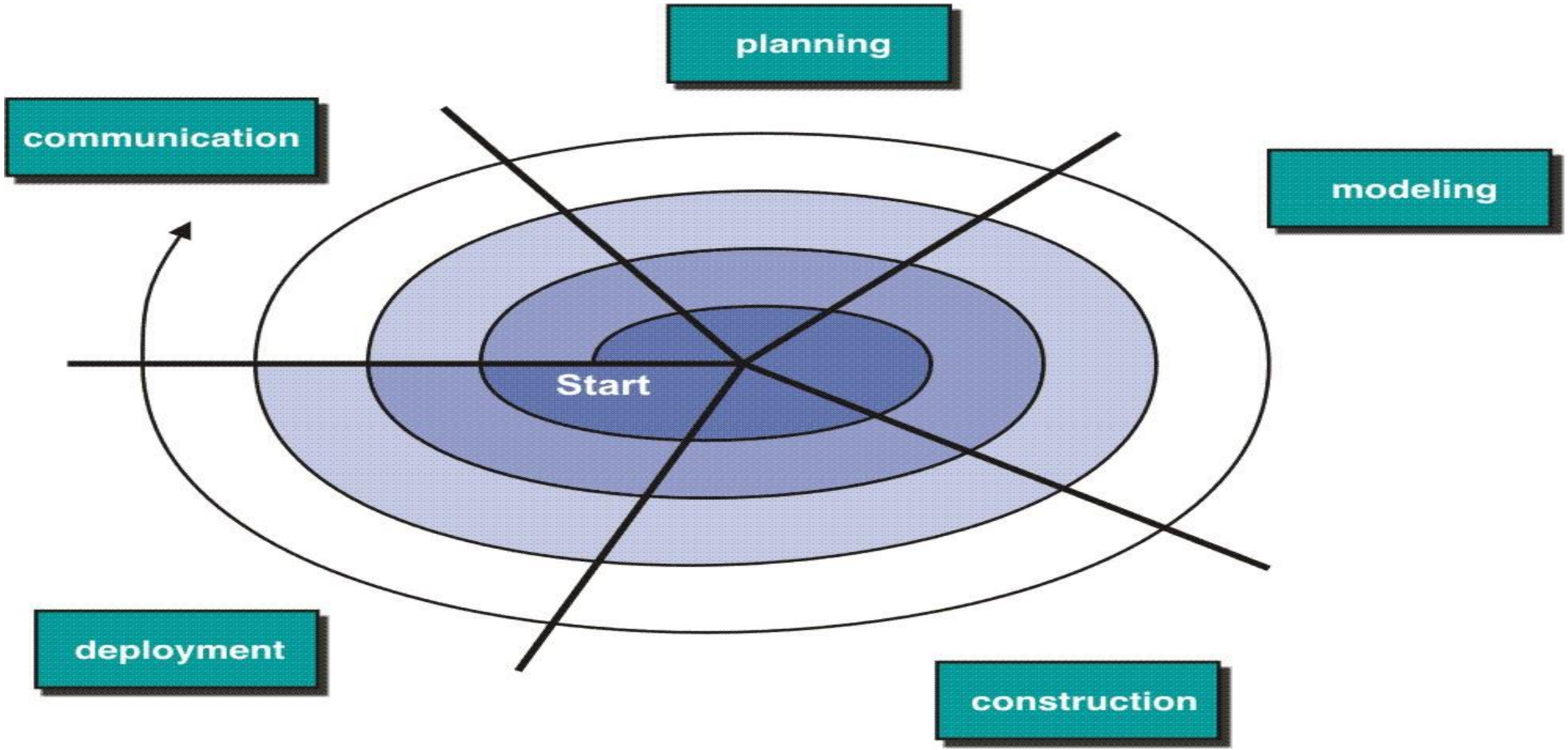
• one of the most important Software Development Life Cycle models, which provides support for Risk Handling.

• It looks like a spiral with many loops.

• The exact number of loops of the spiral is unknown and can vary from project to project.

• Each loop of the spiral is called a Phase of the software development process.

• The exact number of phases needed to develop the product can be varied by the project manager depending upon the project risks.


planning
communication
modeling
Start
deployment
construction

1. communication,
2. planning
    - -estimation
    - -scheduling
    - -risk analysis
3. modeling
    - -analysis
    - -design
4. construction
    - -code
    - -test
5. deployment
    - -delivery
    - -feedback

# Advantages of Spiral Model:

- Risk Handling
- Good for large projects
- Flexibility in Requirements
- Customer Satisfaction

# Disadvantages of Spiral Model:

- **Complex:** more complex than other SDLC models

- **Expensive:** not suitable for small projects as it is expensive

- **Too much dependable on Risk Analysis:** The successful completion of the project is very much dependent on Risk Analysis.

- **Difficulty in time management:** As the number of phases is unknown at the start of the project, so time estimation is very difficult.

# 3.The concurrent development model:

- Also called as concurrent engineering.

- Communication activity has completed its first iteration and exits in the awaiting changes state.

- modeling activity completed its initial communication and then go to the underdevelopment state.

- If the customer specifies the change in the requirement, then the modeling activity moves from the under development state into the awaiting change state.

**NOTE :** All activities exist concurrently but reside in different states.

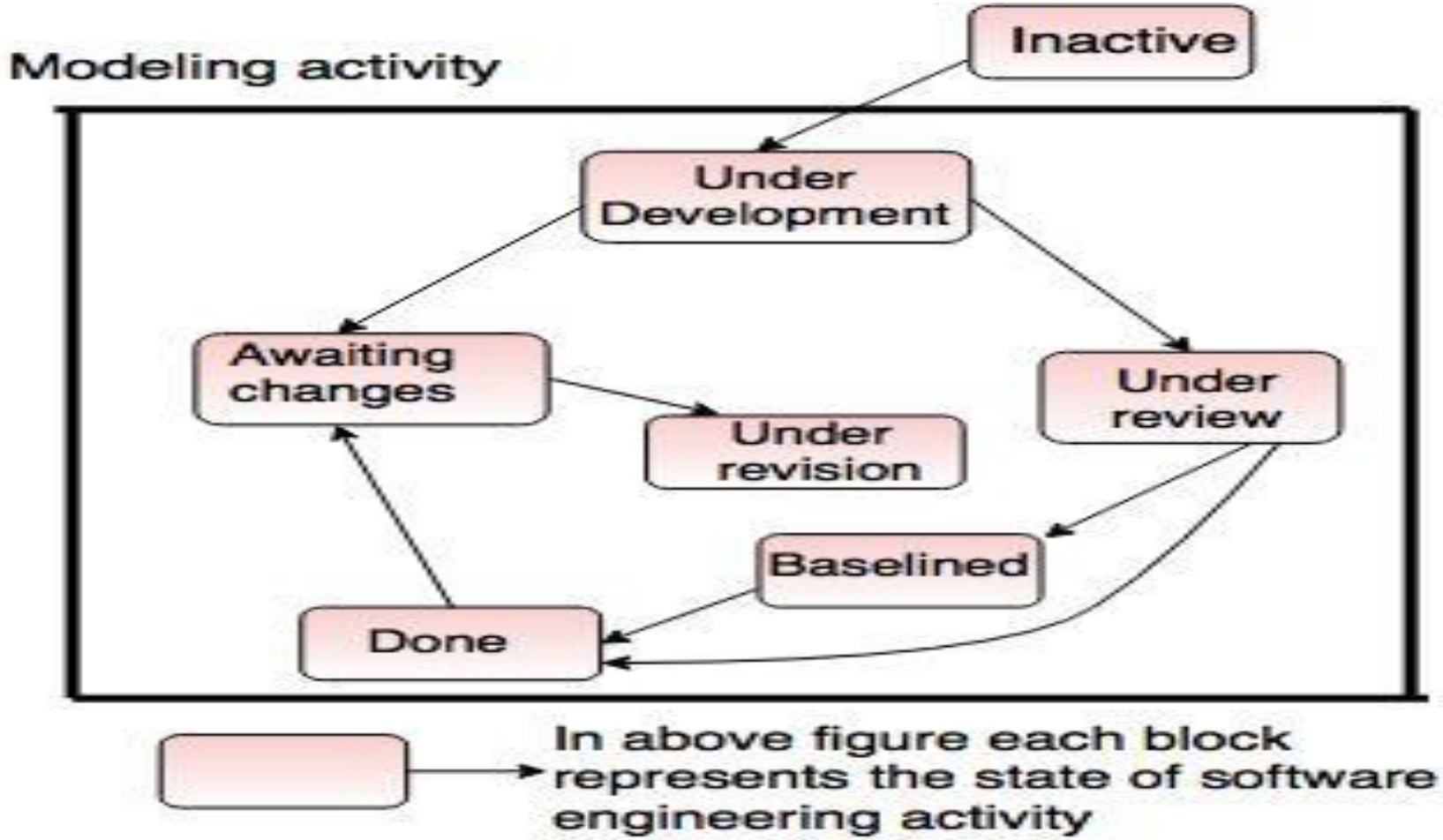



**Fig. - One element of the concurrent process model**

# Advantages of the concurrent development model:

- This model is applicable to all types of software development processes.
- It is easy for understanding and use.
- It gives immediate feedback from testing.
- It provides an accurate picture of the current state of a project.

# Disadvantages of the concurrent development model :

- It needs better communication between the team members.
- This may not be achieved all the time.
- It requires to remember the status of the different activities.

# The Capability Maturity Model Integration (CMMI)

- Developed by the **S/W Engineering Institute (SEI)** at CMU in 1987.

- **Not** a **s/w process model**.

- A **framework** used to **analyze** the **approach** and **techniques** followed by any organization to **develop** a **s/w product**.

- Also provides **guidelines** to enhance the **maturity** of the **s/w products**.

**CMMI represents a process meta-model in two different ways:**

(1) as a continuous model;
(2) as a staged model;

• **Each process area** is defined by CMMI in terms of **"specific goals"** and the **"specific practices"** required to achieve these goals.

• **Specific goals** establish the **characteristics** that must exist if the **activities implied by** a **process area** are to be effective.

• **Specific practices refine** a **goal** into a set of process-related activities.

**Levels of CMMI:**

Level 0: Incomplete
Level 1: Performed
Level 2: Managed
Level 3: Defined
Level 4: Quantitatively
Level 5: Optimized

## Level 0: Incomplete

• The process area (e.g., requirements management) is either **not performed** or **does not achieve all goals and objectives** defined by the CMMI for level 1 capability.

## Level 1 : Performed

1)All of the **specific goals** of the process area (as defined by the CMMI) have been **satisfied**.

2) **Work tasks** required to produce **defined work products** are being conducted.

# Level 2: Managed

•All level 1 criteria have been satisfied.

• All work associated with the process area conforms to an organizationally defined policy.

• All people doing the work have access to adequate resources to get the job done.

• Stakeholders are actively involved in the process area as required.

• Ail(troublesome) work tasks and work products are "monitored, controlled, and reviewed and are evaluated for adherence to the process description" .

## Level 3: Defined

- All **level 2 criteria** have been **achieved**.

- The **process** is "**tailored** from the organization's **set of standard processes** according to the organization's **tailoring guidelines**, and **contributes work products, measures,** and other **process-improvement information** to the organizational **process assets**"

## Level 4: Quantitatively managed

- All **level 3 criteria** have been **achieved**.

- The **process area** is **controlled** and **improved** using **measurement** and **quantitative assessment**.

- "**Quantitative objectives** for **quality** and **process performance** are established and used as criteria in **managing the process**".

## Level 5: Optimized

- All **level 4 criteria** have been **achieved**.

- The **process area** is **adapted** and **optimized** using **quantitative** (statistical) means to meet **changing customer needs** and **to improve the efficacy(ability to produce a desired effect)** of the process area under consideration" .

# Process Patterns

- Process patterns **define a set of activities, actions, work tasks, work products and/or related behaviors**.

- *A **template** is used to **define a pattern**.*

**Types of Patterns:**

- **Task Patterns(task related)**
- **Stage Patterns(activity related)**
- **Phase Patterns(seq. of framework activities)**

**Examples:**

- **Customer communication (a process activity).**
- **Analysis (an action).**
- **Requirements gathering (a process task).**
- **Reviewing a work product (a process task).**
- **Design model (a work product).**

# Process Assessment

•The **process** should be **assessed** to **ensure** that it **meets a set of basic process criteria** that have been shown to be **essential** for a **successful s/w engineering**.

**Many different assessment options are available:**

**1)SCAMPI** (Std CMMI Assessment Method for Process Improvement)

**2)CBA IPI** (CMM-Based Appraisal for Internal Process Improvement)

**3)SPICE(**Software Process Improvement and Capability Determination )

**4)ISO 9001:2000**

# Personal and Team Process Models

## 1)Personal Software Process (PSP)

**Recommends five framework activities:**

1)Planning
2)High-level design
3)High-level design review
4)Development
5)Postmortem

• Stresses the **need** for each s/w engineer to **identify errors** early, to **understand the types of errors.**

# 2)Team Software Process (TSP)

- Each project is **"launched"** using a **"script"** that **defines** the **tasks to be accomplished**.

- Teams are self-directed.

- Measurement is encouraged.

- **Measures** are **analyzed** with the intent of **improving** the **team process**.

# Waterfall Model

- The Waterfall Model was the first Process Model to be introduced.
- It is also referred to as a **linear-sequential life cycle model**.
- It is very simple to understand and use.
- In a waterfall model, each phase must be completed before the next phase can begin and there is no overlapping in the phases.
- The Waterfall model is the earliest SDLC approach that was used for software development.
- The waterfall Model illustrates the software development process in a linear sequential flow.
- This means that any phase in the development process begins only if the previous phase is complete. In this waterfall model, the phases do not overlap.

---

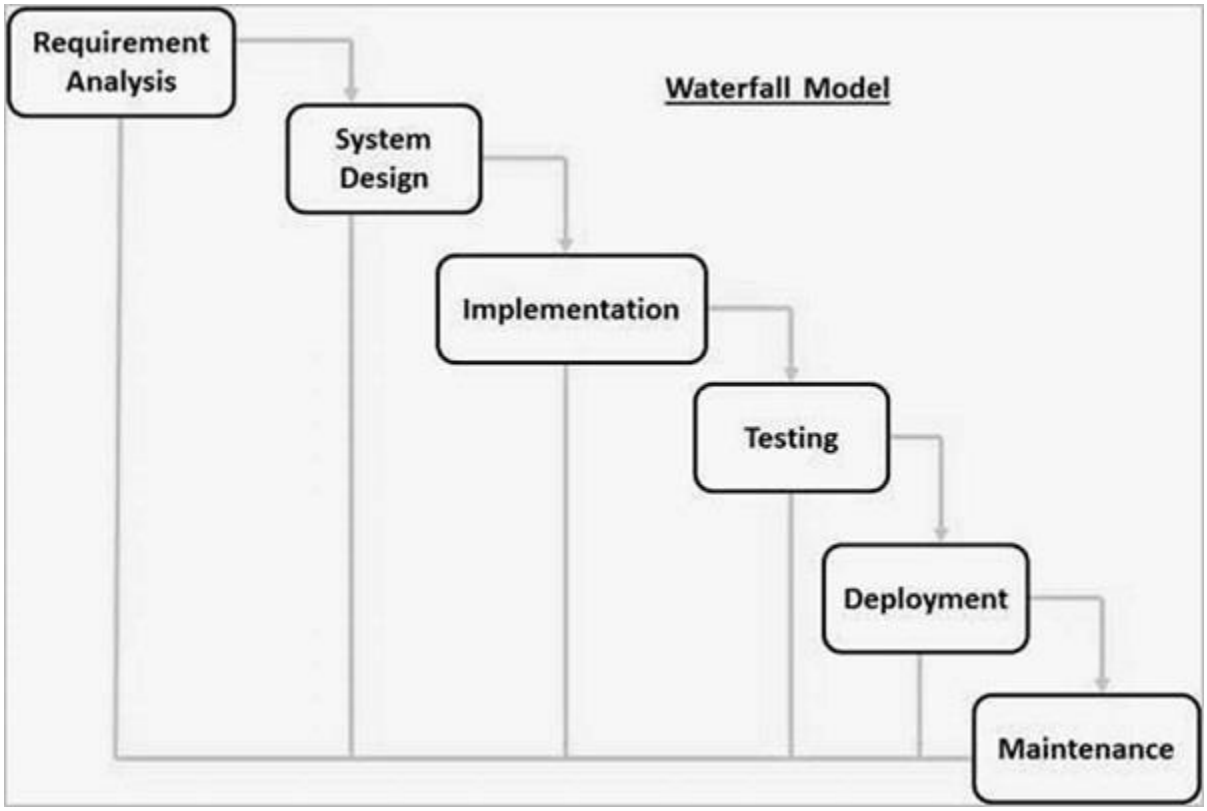
## Waterfall Model - Design

- Waterfall approach was first SDLC Model to be used widely in Software Engineering to ensure success of the project.

- In "The Waterfall" approach, the whole process of software development is divided into separate phases.

- In this Waterfall model, typically, the outcome of one phase acts as the input for the next phase sequentially.

The following illustration is a representation of the different phases of the Waterfall Model.



## The sequential phases in Waterfall model are −

- **Requirement Gathering and analysis** − All possible requirements of the system to be developed are captured in this phase and documented in a requirement specification document.
- **System Design** − The requirement specifications from first phase are studied in this phase and the system design is prepared. This system design helps in specifying hardware and system requirements and helps in defining the overall system architecture.
- **Implementation** − With inputs from the system design, the system is first developed in small programs called units, which are integrated in the next phase. Each unit is developed and tested for its functionality, which is referred to as Unit Testing.
- **Integration and Testing** − All the units developed in the implementation phase are integrated into a system after testing of each unit. Post integration the entire system is tested for any faults and failures.
- **Deployment of system** − Once the functional and non-functional testing is done; the product is deployed in the customer environment or released into the market.
- **Maintenance** − There are some issues which come up in the client environment. To fix those issues, patches are released. Also to enhance the product some better versions are released. Maintenance is done to deliver these changes in the customer environment.
- All these phases are cascaded to each other in which progress is seen as flowing steadily downwards (like a waterfall) through the phases.

- The next phase is started only after the defined set of goals are achieved for previous phase and it is signed off, so the name "Waterfall Model". In this model, phases do not overlap.

## Waterfall Model - Application

Every software developed is different and requires a suitable SDLC approach to be followed based on the internal and external factors. Some situations where the use of Waterfall model is most appropriate are −

- **Requirements are very well documented, clear and fixed.**
- **Product definition is stable.**
- **Technology is understood and is not dynamic.**
- **There are no ambiguous requirements.**
- **Ample resources with required expertise are available to support the product.**
- **The project is short.**

## Waterfall Model - Advantages

- The advantages of waterfall development are that it allows for departmentalization and control.

- A schedule can be set with deadlines for each stage of development and a product can proceed through the development process model phases one by one.

- Development moves from concept, through design, implementation, testing, installation, troubleshooting, and ends

up at operation and maintenance. Each phase of development proceeds in strict order.

**Some of the major advantages of the Waterfall Model are as follows –**

- **Simple and easy to understand and use**
- **Easy to manage due to the rigidity of the model. Each phase has specific deliverables and a review process.**
- **Phases are processed and completed one at a time.**
- **Works well for smaller projects where requirements are very well understood.**
- **Clearly defined stages.**
- **Well understood milestones.**
- **Easy to arrange tasks.**
- **Process and results are well documented.**

## Waterfall Model - Disadvantages

- The disadvantage of waterfall development is that it does not allow much reflection or revision.

- Once an application is in the testing stage, it is very difficult to go back and change something that was not well-documented or thought upon in the concept stage.

## The major disadvantages of the Waterfall Model are as follows –

- **No working software is produced until late during the life cycle.**
- **High amounts of risk and uncertainty.**
- **Not a good model for complex and object-oriented projects.**
- **Poor model for long and ongoing projects.**
- **Not suitable for the projects where requirements are at a moderate to high risk of changing. So, risk and uncertainty is high with this process model.**
- **It is difficult to measure progress within stages.**
- **Cannot accommodate changing requirements.**
- **Adjusting scope during the life cycle can end a project.**
- **Integration is done as a "big-bang. at the very end, which doesn't allow identifying any technological or business bottleneck or challenges early.**

# An Agile View of Process

**Agile-Quick moving, nimble(light), active**

- **Developed to overcome drawbacks of conventional s/w engg.**
- **Not applicable to all projects, products, people and situations.**

**Agile software engineering-**

**Philosophy**

**+**

**set of development guidelines.**

## Philosophy:

- customer satisfaction and early incremental delivery of s/w;
- small, highly motivated project teams;
- informal methods;
- minimal s/w engg work products;
- overall development simplicity.

## set of development guidelines:

- stress delivery over analysis and design.
- active and continuous communication b/n developers and customers.

## Who does it?

- S/w engineers and other project stakeholders (managers, customers, end-users)

## Why is it important?

- fast-paced.
- ever-changing.
- alternative to conventional s/w engg.
- delivers successful systems quickly.

## Also known as "software engineering lite".

**How do we ensure that we've done it right?**

- **If the agile team agrees that the process works and the team produces deliverable s/w increments that satisfy the customer, we've done it right.**

**Agility: An agile team is a nimble team able to appropriately respond to changes**

- **Changes in the s/w being built,**
- **Changes to the team members,**
- **Changes because of new technology,**
- **Changes in project that creates product.**

**Agility is dynamic, content specific, aggressively change embracing, and growth oriented.**

## 12 principles for those who want to achieve agility:

1.Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.

2. welcome changing requirements, even late in development. Agile processes support change for the customer's advantage.

3. Deliver working s/w frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.

4. Business people and developers must work together daily throughout the project.

5. Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.

6. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.

7. Working s/w is the primary measure of progress.

8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.

9. Continuous attention to technical excellence and good design enhances agility.

10. Simplicity is essential.

1 1 . The best architectures, requirements, and designs emerge from self-organizing teams.

12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

## What Is an Agile Process?

- Agile software process three key assumptions.

1 . It is difficult to predict in advance which s/w requirements will persist and which will change. It is equally difficult to predict how customer priorities will change as a project proceeds.

2. For many types of s/w, design and construction are interleaved and It is difficult to predict how much design is necessary before construction is used to prove the design.

3. Analysis, design, construction, and testing are not as predictable as we might like.

## How do we create a process that can manage unpredictability?

- **Process adaptability i.e an agile process must be adaptable and adaptability must be in the form of incremental process.(Done with customer f/b so that appropriate adaptions can be made)**

# The Politics of Agile Development

**1.What is the best way to achieve agility?**

- **Adapting good software engineering concepts.**

**2. How do we build software that meets customers' needs today and exhibits the quality characteristics that will enable it to be extended and scaled to meet customers' needs over the long term?**

- **Adapting good software engineering concepts**

## Human Factors:

- **Agile development focuses on the talents and skills of individuals, molding the process to specific people and teams.**

**Process molds to the needs of the people and team**

# List of the human factors:

1. Competence.
2. Common focus.
3. Collaboration.
4. Decision-making ability.
5. Fuzzy problem-solving ability.
6. Mutual trust and respect.
7. Self-organization.

## Agile Process Models(7):

1. Extreme Programming(XP)
2. Adaptive s/w Development(ASD)
3. Dynamic s/w Development Method(DSDM)
4. Scrum
5. Crystal
6. Feature Driven Development(FDD)
7. Agile modeling(AM)

# I. Extreme Programming (XP)

- most widely used agile process, originally proposed by Kent Beck

- Uses an object-oriented approach.

- Describes a set of rules for framework activities:

- planning,
- design,
- coding, and
- testing.

- **XP Planning**

**-Begins with the creation of user stories.**

**-Agile team assesses each story and assigns a cost.**

**-Stories are grouped to for a deliverable increment.**

**-A commitment (agreement on stories to be included, delivery date, and other project matters) is made on delivery date.**

**-After the first increment project velocity (no. of customer stories implemented during the first release) is used to help define subsequent delivery dates for other increments.**

- **XP Design**

**-Follows the KIS principle.**

**-Encourage the use of CRC cards(class-responsibility-collaborator) cards identify and organize the object-oriented classes.**

**-For difficult design problems, suggests the creation of spike solutions — a design prototype.**

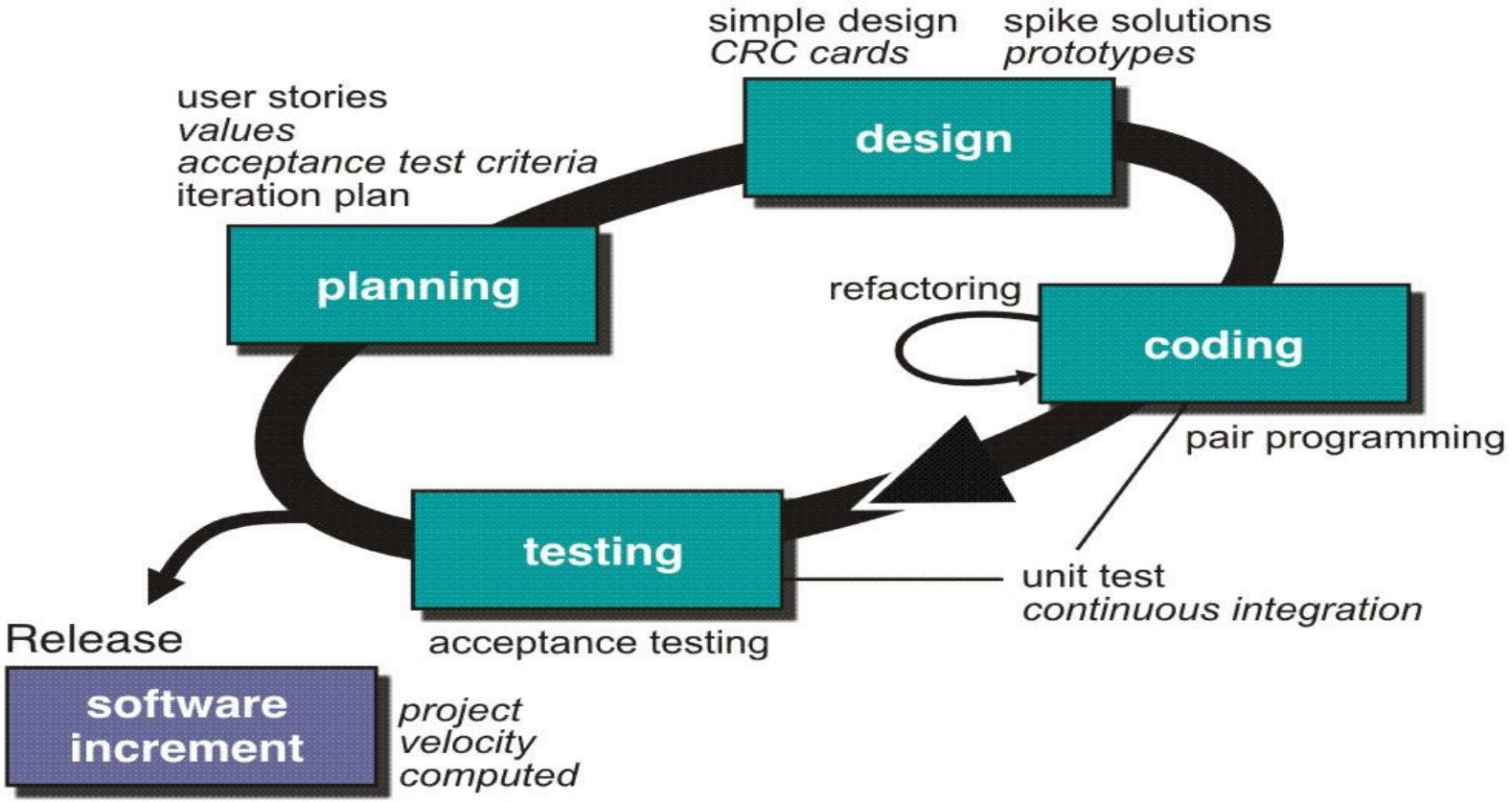
**-Encourages refactoring — an iterative refinement of the internal program design.**

- **XP Coding**

**-Recommends the construction of a unit test for a store *before* coding commences.**
**-Encourages pair programming.**

- **XP Testing**

**-All unit tests are executed daily.**
**-Acceptance tests(Customer tests) are defined by the customer and executed to assess customer visible functionality.**


simple design
CRC cards
spike solutions
prototypes
user stories
values
acceptance test criteria
iteration plan
design
planning
refactoring
coding
pair programming
testing
unit test
continuous integration
Release
acceptance testing
software increment
project velocity computed

# II.Adaptive Software Development (ASD)

- Proposed by Jim Highsmith.
- technique for building complex s/w and systems.
- focus on human collaboration and team self-organization.

## ASD life cycle incorporates three phases:

1.speculation,
2.collaboration, and
3.learning

# 1. Speculation:

- **Project is initiated and adaptive cycle planning is conducted.**
- **Adaptive cycle planning uses**

**(i) project initiation info—the *customer's mission statement, project constraints* (e.g., delivery dates or user descriptions),**

**+**

**(ii) basic requirements—to define the *set of release cycles* (software increments) that will be required for the project.**

## 2. Collaboration:

- **Motivated people work together to multiply their talent and creative o/p beyond their absolute numbers.**

- **People working together must trust one another to:**

**(1) Criticize(find fault) without animosity( enemity, hatred);**
**(2) Assist without resentment(anger);**
**(3) work as hard or harder as they do;**
**(4) have the skill set to contribute to the work at hand; and**
**(5) communicate problems or concerns in a way that leads to effective action.**

## 3. Learning:

- **ASD team begin to develop the components that are part of an adaptive cycle, the emphasis is on learning as much as it is on progress toward a completed cycle.**

- **Highsmith argues that s/w developers often over estimate their own understanding *(of the technology, the process, and the project)* and that learning will help them to improve their level of real understanding.**

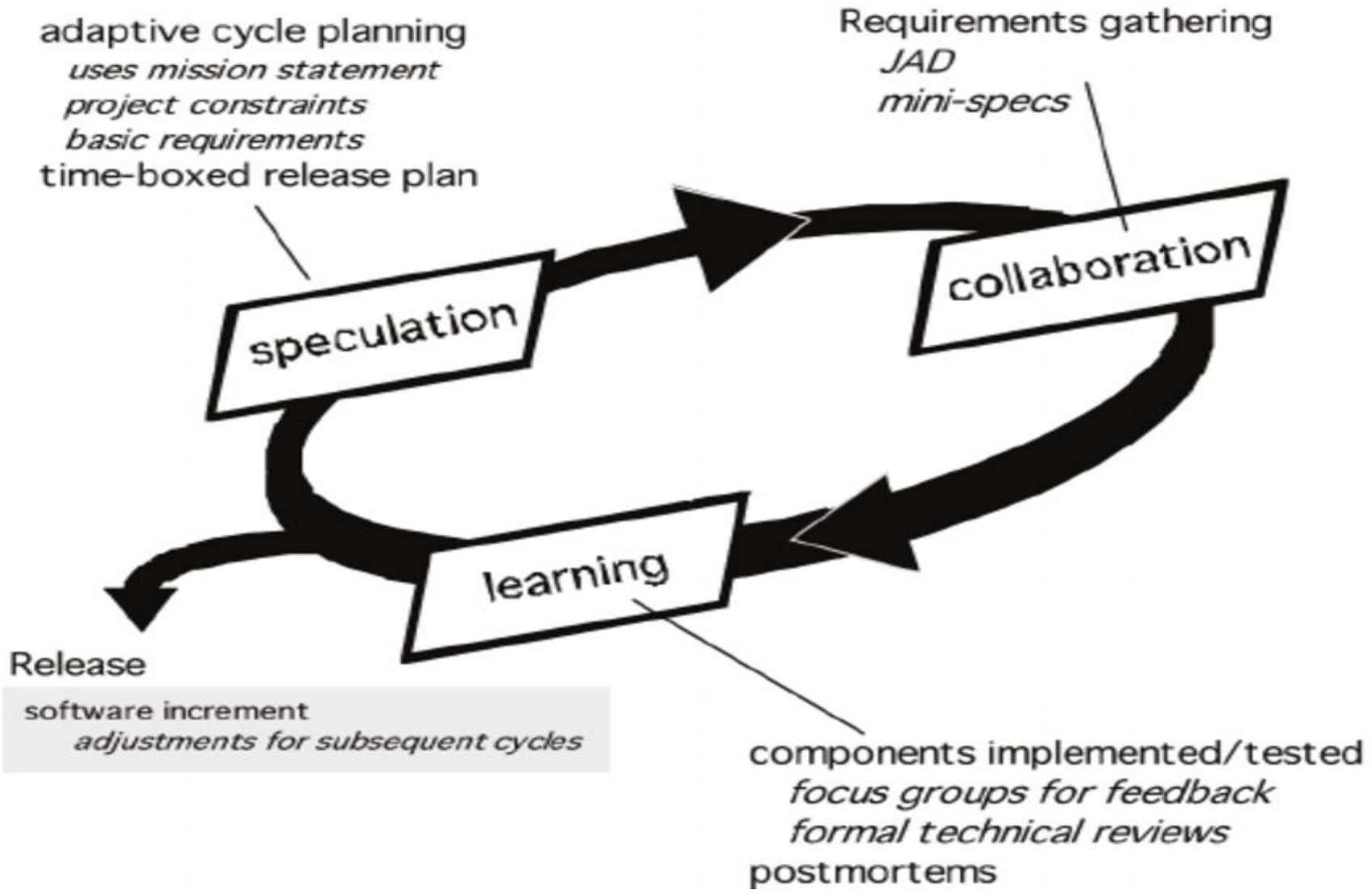
## ASD teams learn in three ways:

**1 . Focus groups: customer and/or end-users provide f/b on s/w increments that are being delivered. This provides a direct indication of whether or not the product is satisfying business needs.**

**2. Formal technical reviews: Team members review the s/w components that are developed, improving quality and learning as they proceed.**

**3. Postmortems: Team becomes introspective(self examination), addressing its own performance and process.**

# Fig 4.2 ASD(from TB)



## III. Dynamic Systems Development Method (DSDM):

- **Provides a framework for building and maintaining systems which meet tight time constraints through the use of incremental prototyping in a controlled project environment.**

- **Pareto principle- 80% of an application(only enough work is required for each increment to facilitate movement to the next increment) can be delivered in 20% of the time it would take to deliver the complete (100 percent) application.**

## DSDM life cycle:

- Defines 3 different iterative cycles, preceded by 2 additional life cycle activities:

1.Feasibility study—establishes the basic business requirements and constraints associated with the application to be built and then assesses whether the application is a viable candidate for the DSDM process.

**2.Business study—establishes the functional and information requirements that will allow the application to provide business value;**
**also, defines the basic application architecture and identifies the maintainability requirements for the application.**

**3.Functional model iteration—produces a set of incremental prototypes that demonstrate functionality for the customer.**

- **The intent during this iterative cycle is to gather additional requirements by eliciting feedback from users as they exercise the prototype.**

**4.Design and build iteration— revisits prototypes built during the functional model iteration to ensure that each has been engineered in a manner that will enable it to provide operational business value for end-users.**

- **In some cases, the functional model iteration and the design and build iteration occur concurrently.**

**5.Implementation—places the latest s/w increment into the operational environment.**

- **It should be noted that:**

**(1) the increment may not be 100% complete**

**or**

**(2) changes may be requested as the increment is put into place.**

- **In either case, DSDM development work continues by returning to the function model iteration activity.**

## IV. Scrum (name derived from an activity that occurs during a rugby match)

**Activity-A group of players forms around the ball and the teammates work together (sometimes violently!) to move the ball downfield.**

- developed by Jeff Sutherland and his team in the early 1990s.
- further development of the Scrum methods has been performed by Schwaber and Beedle.

**Scrum principles(6) are consistent (dependable, compatable with the agile manifesto:**

**1) Small working teams are organized to "maximize communication, minimize overhead, and maximize sharing of tacit, informal knowledge."**

**2) The process must be adaptable to both technical and business changes "to ensure the best possible product is produced.“**

**3) The process yields frequent software increments "that can be inspected, adjusted, tested, documented, and built on."**

4) Development work and the people who perform it are partitioned "into clean, low coupling partitions, or packets.“

5) Constant testing and documentation is performed as the product is built.

6) The Scrum process provides the "ability to declare a product 'done' whenever required.

• **Scrum principles are used to guide development activities within a process that incorporates the following framework activities(5):**

- **requirements,**
- **analysis,**
- **design,**
- **evolution, and**
- **delivery.**

- **Within each framework activity, work tasks occur within a process pattern called a sprint.**

Fig 4.3:Scrum process flow(from TB)

• Backlog—a prioritized list of project requirements or features that provide business value for the customer. Items can be added to the backlog at any time (this is how changes are introduced).

• Sprints— work units that are required to achieve a requirement defined in the backlog that must be fit into a predefined time-box (typically 30 days).

•During the sprint, the backlog items that the sprint work units address are frozen
(i.e., changes are not introduced during the sprint).

• Scrum meetings—are short (typically 15 minutes) meetings held daily by the Scrum team.

Three key questions are asked and answered by all team members:

1) What did you do since the last team meeting?
2) What obstacles are you encountering?
3) What do you plan to accomplish by the next team meeting?

A team leader, called a "Scrum master," leads the meeting and assesses the responses from each person.

- **Demos—deliver the s/w increment to the customer so that functionality that has been implemented can be demonstrated and evaluated by the customer.**